

avito.tech 

СТРУКТУРНЫЕ ПАТТЕРНЫ

Афанасьев Юра

Образуют из классов
и объектов более
крупные структуры

Решаемые задачи

Используя несколько небольших классов возможно

- создать большие структуры
- расширить возможности базового класса

Наследование
используется
для составления
композиций
из интерфейсов
и реализаций

1. **Adapter**. Адаптер

2. **Bridge**. Мост

3. **Composite**. Компоновщик

4. **Decorator**. Декоратор

5. **Facade**. Фасад

6. **Flyweight**. Приспособленец

7. **Proxy**. Заместитель

ADAPTER



АДАПТЕР

Преобразует интерфейс
одного класса
в интерфейс другого,
который ожидают клиенты

Обеспечивает совместную работу
классов с несовместимыми
интерфейсами, которая
без него была бы невозможна

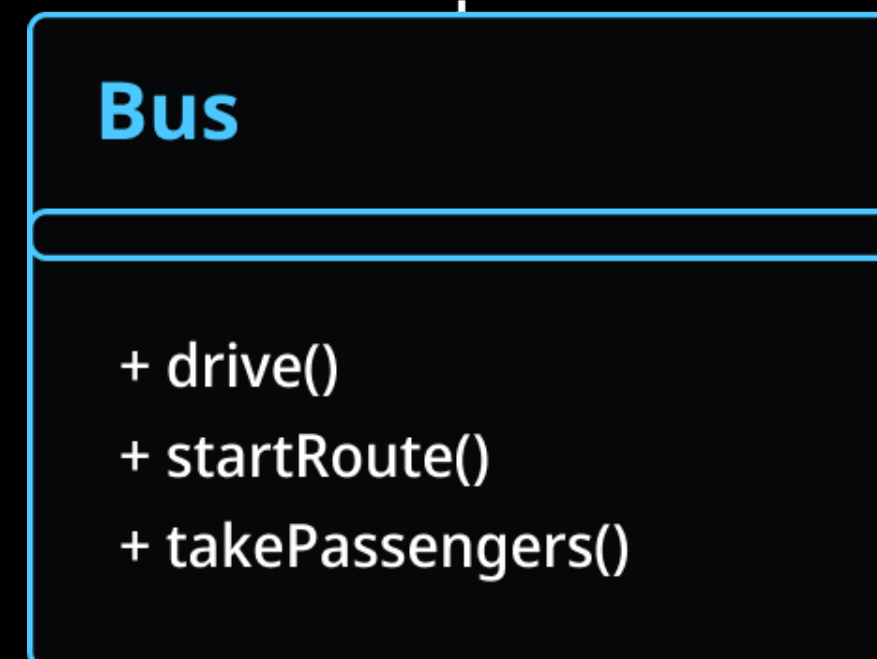
Решаемая задача

- Позволяет использовать классы с разными интерфейсами совместно
- Добавляет классу интерфейс, позволяя его использовать в другой иерархии интерфейсов

Общественный транспорт



Сущность автобуса



ITruck

- + drive()
- + startRoute()
- + takeGoods()

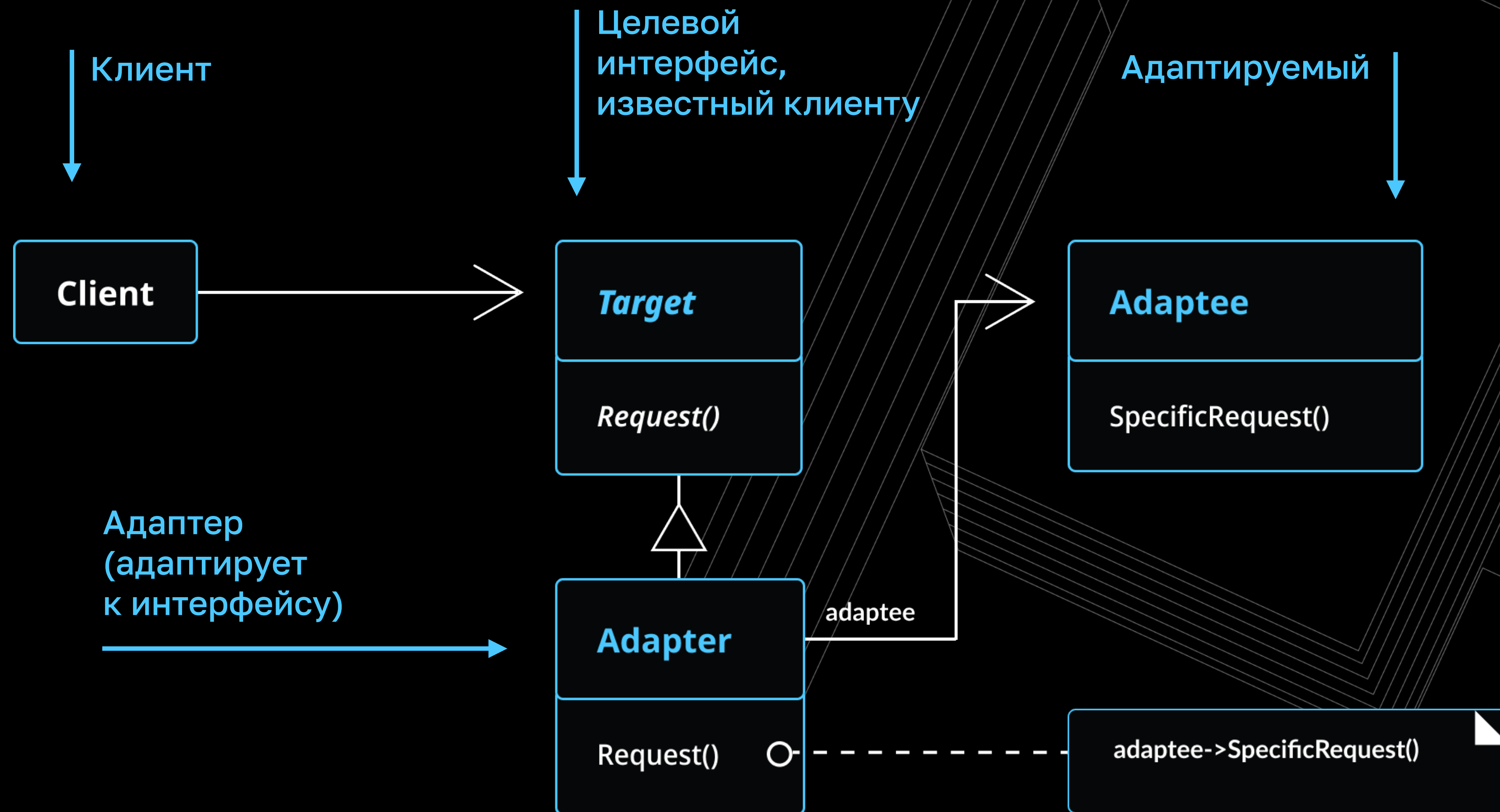
Грузовик

Как
удовлетворять
двум интерфейсам



Когда понадобится?

- Расширение библиотеки из vendor
- Код, который вы не можете менять
- Класс должен использоваться в двух иерархиях одновременно



Знает только
о целевом
интерфейсе

Клиент

Целевой интерфейс

```
class Client
{
    public function method(ITarget $target)
    {
        // Некоторая бизнес-логика

        $target->request();
    }
}
```

Его методы не совпадают
с методами адаптируемого
класса

Целевой интерфейс

```
interface ITarget
{
    public function request();

    // public function anotherRequest();
}
```

Другие методы
интерфейса

Адаптируемый
класс

```
class Adaptee
{
    public function specialRequest()
    {
        // ...КОД...
    }

    // public function anotherSpecialRequest() {}
}
```

Класс может
содержать любое
количество методов

Приводит адаптируемый
класс к целевому
интерфейсу

Адаптер

Целевой
интерфейс

Адаптируемый
класс

Проксируем вызовы
методов и реализуем
доп.бизнес-логику

Реализуем все
методы интерфейса

```
class Adapter implements ITarget
{
    private Adaptee $adaptee;

    public function __construct(Adaptee $adaptee)
    {
        $this->adaptee = $adaptee;
    }

    public function request()
    {
        // по надобности производятся дополнительные действия

        $this->adaptee->specialRequest();
    }

    // public function anotherRequest() {}
}
```

Адаптируемый
класс

```
function testAdapter()  
{  
    $adapter = new Adapter(new Adaptee());  
  
    (new Client())->method($adapter);  
}
```

Оборачиваем в адаптер

Передаём адаптер
в класс клиента

Плюсы

- Приводит класс к отличному от своего интерфейса
- Позволяет включить класс в систему, спроектированную для классов под другой интерфейс
- Позволяет адаптеру заменить или реализовать некоторые операции адаптируемого класса

Адаптер

Заменяем или
дополняем операции
адаптируемого класса

```
class Adapter implements Target
{
    private Adaptee $adaptee;

    public function __construct(Adaptee $adaptee)
    {
        $this->adaptee = $adaptee;
    }

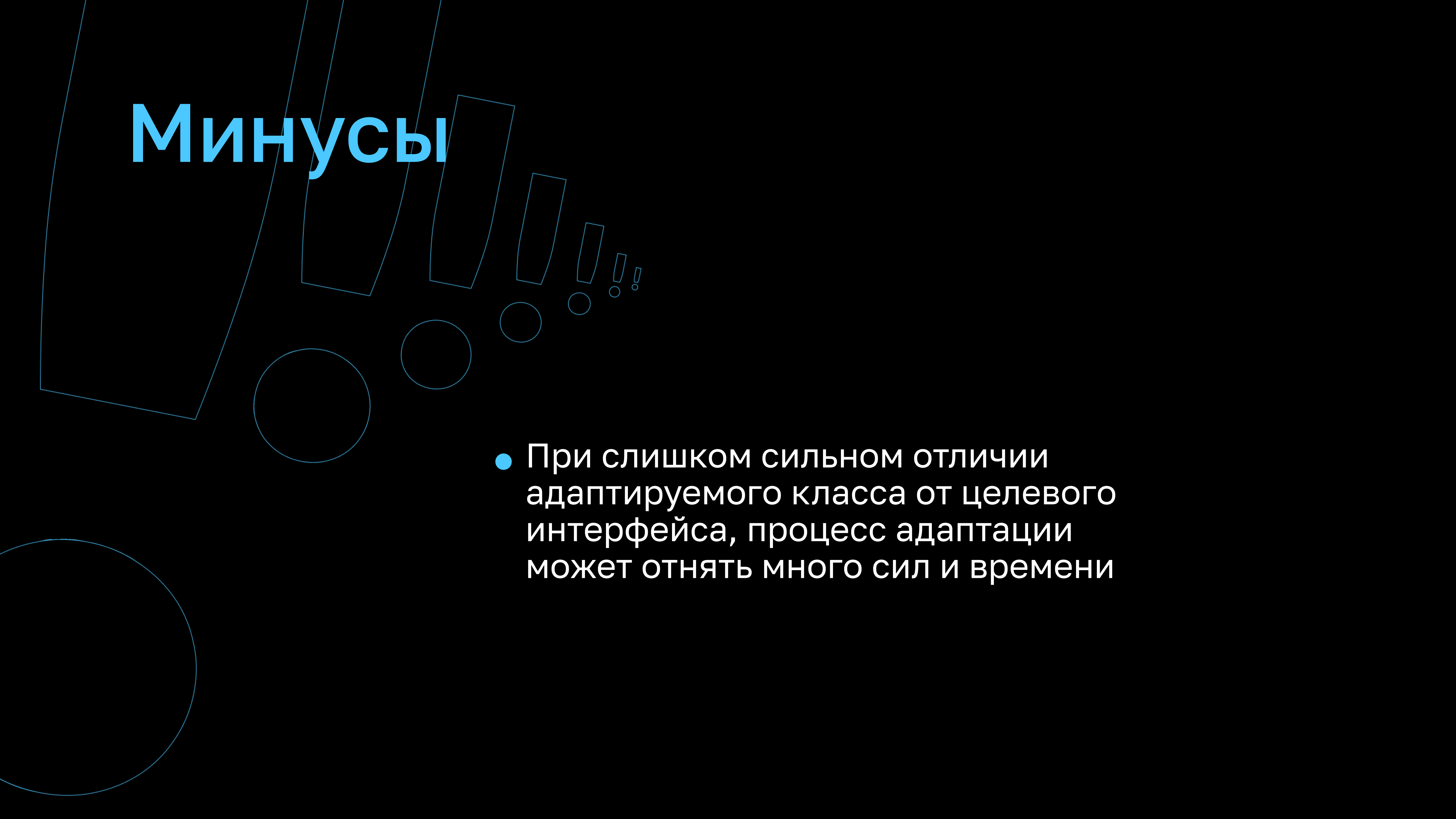
    public function request(): array
    {
        if (!$something) {
            return [];
        }

        $items = $this->adaptee->specialRequest();

        $result = [];
        foreach ($items as $item) {
            $result[$item->getId()] = $item->getData();
        }

        return $result;
    }
}
```

Минусы

The background features several abstract geometric shapes in a light blue color. These include a large rectangle on the left, a series of smaller rectangles and circles arranged in a descending diagonal line towards the center, and a large circle in the bottom left corner.

- При слишком сильном отличии адаптируемого класса от целевого интерфейса, процесс адаптации может отнять много сил и времени

Общественный транспорт

IPublicTransport

+ drive()
+ takePassengers()
+ stop()

Bus

+ drive()
+ startRoute()
+ stop()

IAirplane

+ fly()
+ takePassengers()
+ arrival()

Самолёт

Адаптер будет
громоздким



Создание класса-обёртки с требуемым интерфейсом

A red car body is shown in the center, surrounded by a vast array of its disassembled parts. The parts are laid out on a white surface, including the front and rear doors, the hood, the trunk lid, the front and rear seats, the dashboard, the steering wheel, the engine block, the transmission, the wheels, the suspension components, the brakes, the lights, and numerous smaller mechanical and electrical parts. The car body is red, while the seats are grey. The background is a plain, light-colored surface. The word 'KOMPONENT' is written in large, blue, stylized letters across the bottom of the image.

КОМПОНОВЩИК

Компонует объекты
в древовидные структуры
для представления иерархий
часть-целое

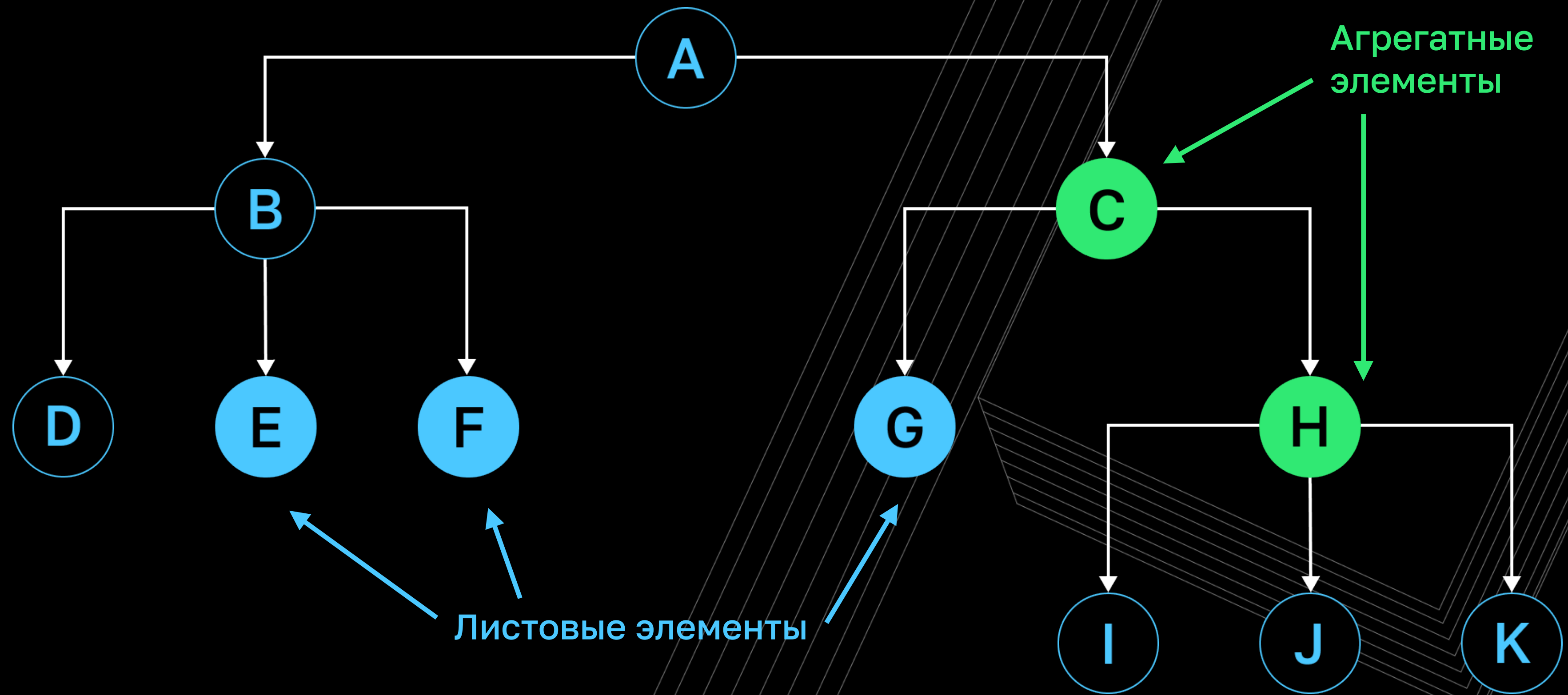
Позволяет клиентам
единообразно трактовать
индивидуальные
и составные объекты

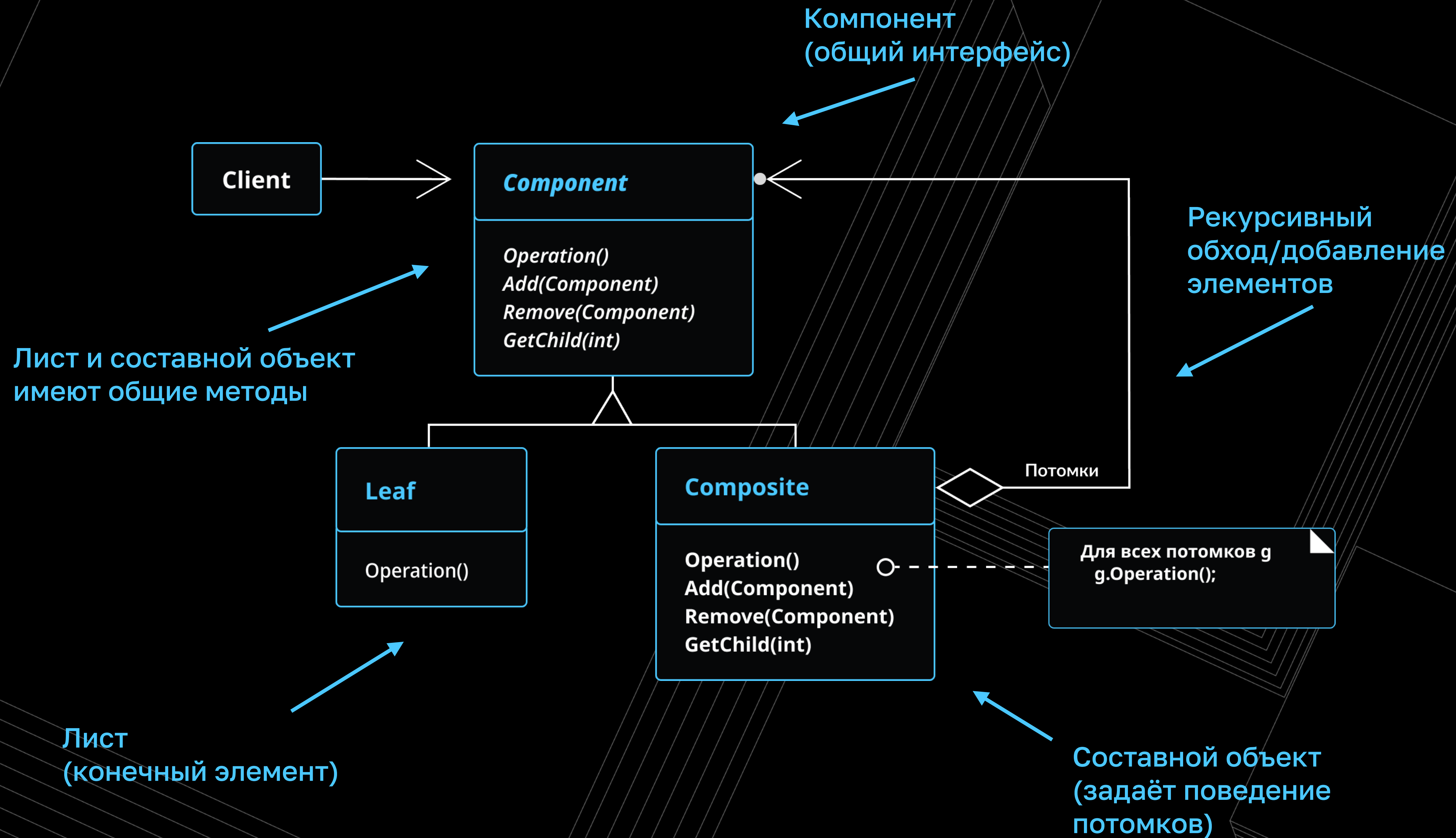
Решаемая задача

- Предоставляет иерархию объектов вида часть-целое
- Клиенты единообразно трактуют составные и агрегатные объекты

целое – то, что состоит из частей

часть – элемент некоторого целого





Операция выполняющая
действие с деревом

Компонент
(унифицирует методы
листов и агрегатов)

```
abstract class Component
{
    abstract public function operation();

    // abstract public function anotherOperation();

    public function add(Component $component): self
    {
        throw new RuntimeException();
    }

    public function remove(Component $component): self
    {
        throw new RuntimeException();
    }

    public function getChild(Component $component): self
    {
        throw new RuntimeException();
    }
}
```

Создаём необходимое
количество действий

Методы для работы
с деревом
(переопределяются
в агрегате)

Листовой элемент
(конечный элемент
в дереве)

Реализует только
методы операций
(методы манипуляции
не наследуются)

```
class Leaf extends Component
{
    public function operation()
    {
        // Реализация операций

        return 'Листовой элемент.' . PHP_EOL;
    }
}
```

Составной объект
(ветвь дерева;
не конечный
элемент)

```
class Composite extends Component
{
    private \SplObjectStorage $elements;

    public function __construct()
    {
        $this->elements = new \SplObjectStorage();
    }

    public function operation()
    {
        $result = '';
        foreach ($this->elements as $element) {
            $result .= $element->operation();
        }
        return $result;
    }

    public function add(Component $element): self
    {
        $this->elements->attach($element);
        return $this;
    }

    public function remove(Component $element): self
    {
        $this->elements->detach($element);
        return $this;
    }
}
```

Для обхода дерева,
итеративно проходим
по элементам

Реализация методов
для работы с деревом

Конструирование
дерева

```
function testComposite()
{
    $composite1 = (new Composite())
                ->add(new Leaf())
                ->add(new Leaf());

    $composite2 = (new Composite())
                ->add(
                    (new Composite())
                    ->add(new Leaf())
                )
                ->add(new Leaf());

    $root = (new Composite())
            ->add($composite1)
            ->add($composite2);

    echo $root->operation();
}
```

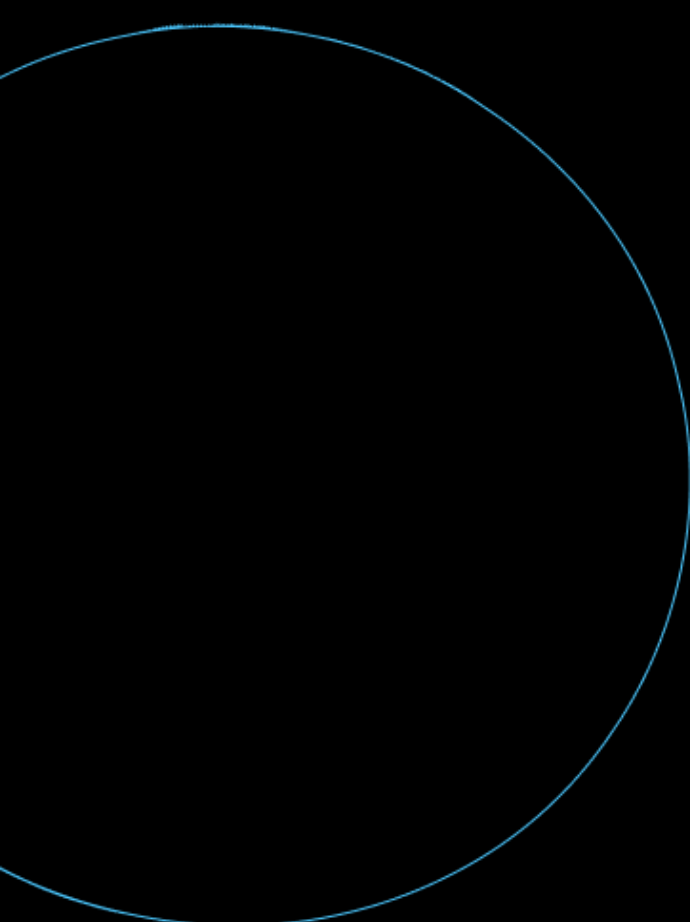
Выполняем операции
для работы с деревом

Плюсы

- Определяет иерархию классов, состоящие из примитивных и составных элементов
- Упрощает архитектуру клиента
- Облегчает добавление новых видов компонентов (агрегата или листа)

Минусы



- Трудно наложить ограничения на то, какие объекты могут ВХОДИТЬ В СОСТАВ КОМПОЗИЦИИ
- 

```
class Composite implements Component
{
    private $components;

    public function add(Component $component): void
    {
        if (!$component instanceof SpecialLeaf
            || !$component instanceof SpecialComposite
        ) {
            throw new LogicalException(
                'Illegal leaf ' . get_class($component)
            );
        }

        $this->components[] = $component;
    }
}
```

Наложение
ограничений
на входные классы



Описывает рекурсивную
композицию так,
чтобы клиенту не
пришлось проводить
различие между
простыми и составными
объектами

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

